

Graded Project - Part 2

AI Project Technical Methodologies

E-commerce customer churn prediction, built with production engineering practice

Implementation of the churn project framed in Part 1: Git and GitHub for versioning and CI, a cookiecutter project structure with one script per pipeline step, PEP8 tooling, Poetry for reproducibility, Sphinx documentation, and MLflow for experiment tracking, project packaging, model registry and serving.

Code repository

https://github.com/ribbzz/AI_Project_Methodology

Group members

- Rabeeh Abou Ismail
- Amjad Bsar
- Majd Hammoud

Dataset	E-Commerce Customer Churn (Kaggle) - 5,630 customers
Deliverable	Part 2 report - group dedicated repository
Date	September 2026

1. Summary

Part 1 of this project framed a customer churn prediction initiative for RetailGenius. Part 2 implements that framing as a working machine learning project on the E-Commerce Customer Churn dataset, with the emphasis the brief asks for: **the engineering practice around the model, not the model's score.**

Code repository

https://github.com/ribbzz/AI_Project_Methodology - the full project, including the dataset, the generated models and the MLflow tracking store, is committed there. Every command in this report can be run from a fresh clone.

Requirements of the brief and where they are met

Brief requirement	Implementation	In the repository
Git for code and model versioning, GitHub as the collaboration platform	Ten focused commits; GitHub Actions runs formatting, linting, tests and the documentation build on every push; models are versioned in the MLflow registry	git history, .github/workflows/ci.yml
Separate scripts for data preparation, feature engineering, training and inference	Four modules, each runnable on its own and exposed as a Make target, a Poetry script and an MLflow entry point	src/churn/data, features, models
Cookiecutter project template	Cookiecutter Data Science layout, adapted to a src/ package	repository root
Same Python version, PEP8 style	Python 3.11+ declared for the group; black, flake8 and pylint configured and passing (pylint 10.00/10)	pyproject.toml, setup.cfg, .pylintrc
Dependency management	Poetry project, with pip requirement files and an MLflow python_env kept in step	pyproject.toml, requirements*.txt, python_env.yaml
Automated documentation (Sphinx)	Sphinx with autodoc and napoleon, generated from Google-style docstrings	docs/source, make docs
MLflow tracking - parameters, metrics, multiple runs in the UI	9 training runs and 1 sweep parent run, each with parameters, 8 metrics, 4 plots and the model	mlflow.db, mlruns/
MLflow Projects packaging	MLproject file with seven entry points	MLproject, python_env.yaml
MLflow Model Registry	Best run registered automatically as ecommerce-churn-classifier, alias champion	train_model.register_best
Local serving with MLflow	mlflow models serve on the champion; a client script sends real customers to it	make serve, scripts/score_request.py
Deliverables: code, generated models, MLflow outputs	All committed; a helper rewrites the store's absolute paths after a clone	models/, mlruns/, scripts/relocate_mlflow.py

2. Dataset and problem

The dataset is the Kaggle **E-Commerce Customer Churn Analysis and Prediction** workbook: one row per customer, 20 columns, 5,630 customers. The target, **Churn**, is 1 for 948 customers (**16.8%**). The raw file is committed in data/raw/ so that the pipeline runs without a Kaggle account.

Group	Columns
Identifier and target	CustomerID, Churn
Relationship	Tenure, DaySinceLastOrder, OrderCount, OrderAmountHikeFromLastYear, CouponUsed, CashbackAmount
Engagement	HourSpendOnApp, NumberOfDeviceRegistered, PreferredLoginDevice
Service experience	SatisfactionScore, Complain, WarehouseToHome
Profile	Gender, MaritalStatus, CityTier, NumberOfAddress, PreferredPaymentMode, PreferredOrderCat

What the exploratory analysis changed in the pipeline

The notebook notebooks/1.0-eda.ipynb is committed with its outputs. Five findings shaped the code:

Finding	Evidence	Consequence in the code
Imbalanced target	16.8% churners; a model predicting 'no churn' for everyone is 83% accurate	Stratified split; class weights; PR-AUC and lift as the main metrics, accuracy not reported
Missing values	7 numeric columns with 4.5-5.5% missing (DaySinceLastOrder the highest, 307 rows)	Median imputation inside the sklearn pipeline, so training and serving treat gaps identically
Duplicate category spellings	Phone / Mobile Phone, CC / Credit Card, COD / Cash on Delivery, Mobile / Mobile Phone	make_dataset.harmonise_categories merges them before encoding
Strong early-life churn	65.5% of churners have a tenure of 0-1 month; median tenure is 1 for churners vs 10 for the rest	Engineered ratios normalised by tenure (orders, recency) and an IsNewCustomer flag
Service friction matters	Customers who complained churn at 31.7% vs 10.9%; single customers at 26.7%	Kept as raw features; they surface in the importance plot

3. Versioning and collaboration

The project lives in a GitHub repository shared by the group. The history is kept readable on purpose - each commit adds one coherent piece, so a reviewer can follow how the project was built:

#	Commit	Content
1	Set up project structure and tooling	layout, Poetry, requirements, linters, Makefile, config
2	Add raw dataset and data preparation step	data/raw, make_dataset.py
3	Add feature engineering step	build_features.py, fitted preprocessor
4	Add model training with MLflow tracking and registry	train_model.py, evaluate.py, visualize.py
5	Add batch inference and MLflow serving client	predict_model.py, MLproject, serving scripts
6	Add test suite and CI workflow	tests/, .github/workflows/ci.yml
7	Add Sphinx documentation	docs/source
8	Add exploratory data analysis notebook	notebooks/1.0-eda.ipynb
9	Add MLflow runs, registered model and report figures	mlflow.db, mlruns/, reports/
10	Write README	setup, usage, results

What is versioned, and with which tool

Artefact	Tool	Why
Source code, configuration, notebooks, docs	Git / GitHub	History, review and collaboration
Raw dataset (556 KB)	Git	Small and immutable - the pipeline is reproducible from a clone
Intermediate and processed data	Not versioned (git-ignored)	Fully regenerated by make data; committing them would duplicate the source of truth

Artefact	Tool	Why
Trained models	MLflow Model Registry, stored in the repository	Versions, aliases and lineage back to the run that produced each model
Experiments	MLflow tracking store (mlflow.db + mlruns/)	Parameters, metrics and artefacts of every run, viewable in the UI

Continuous integration

GitHub Actions (.github/workflows/ci.yml) runs on every push and pull request, on Python 3.11 - the lower bound of the group standard. It fails the build if any of these fail:

```
black --check src tests scripts      # formatting
flake8 src tests                     # PEP8
pylint src/churn                     # static analysis
pytest --cov=churn                   # 21 tests
sphinx-build -W docs/source docs/build # docs, warnings are errors
```

Recommended team workflow. One feature branch per change, a pull request reviewed by another member, merge only on a green CI run, main protected. The CI makes 'it works on my machine' visible before it reaches the shared branch.

4. Code structure

The layout follows **Cookiecutter Data Science**, adapted in two ways: the code is an installable package under src/ (so tests and scripts import it the same way), and MLflow's store sits at the root where every command can find it.

```
AI_Project_Methodology/
├─ data/
│  ├─ raw/                  E_Commerce_Dataset.xlsx (immutable)
│  ├─ interim/              clean.parquet (generated)
│  └─ processed/            train / test.parquet (generated)
├─ docs/source/             Sphinx: index, usage, API reference
├─ models/                  preprocessor.joblib
├─ mlflow.db, mlruns/        tracking store, artefacts, model registry
├─ notebooks/1.0-eda.ipynb   exploratory analysis
├─ reports/                  figures/, model_comparison.csv, predictions.csv
├─ scripts/                  score_request.py, report_figures.py, relocate_mlflow.py
├─ src/churn/
│  ├─ config.py              every path and constant, defined once
│  ├─ data/make_dataset.py    STEP 1 data preparation
│  ├─ features/build_features.py STEP 2 feature engineering
│  ├─ models/train_model.py   STEP 3 training, tracking, registry
│  ├─ models/predict_model.py STEP 4 batch inference
│  ├─ models/evaluate.py      metrics
│  └─ visualization/visualize.py evaluation plots
├─ tests/                    test_pipeline.py, test_training.py
├─ MLproject, python_env.yaml MLflow project
├─ pyproject.toml            Poetry + black/pytest configuration
├─ requirements.txt, requirements-dev.txt
├─ setup.cfg, .pylintrc      flake8 / pylint configuration
└─ Makefile                   one target per step
```

The four pipeline steps

data/raw -> data/processed

reports/predictions.csv

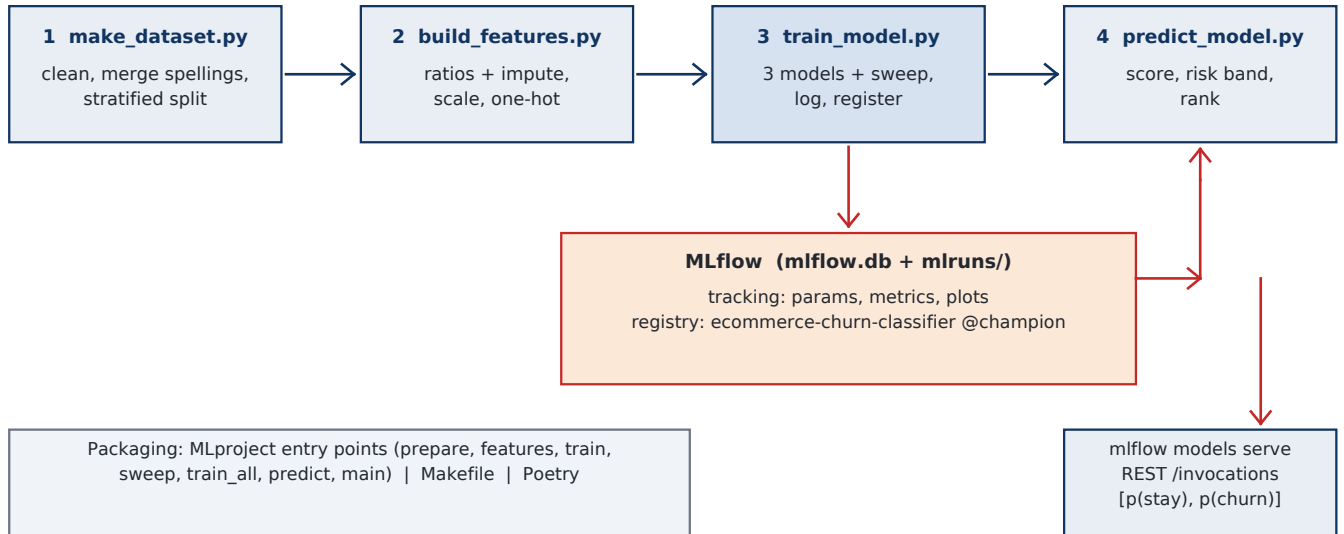


Figure 1 - Pipeline steps, the MLflow store they write to, and the two ways a model leaves it: batch scoring and the REST endpoint.

Step	Module	Input -> output	Key design choice
1 Data preparation	data/make_dataset.py	raw workbook -> clean.parquet, train/test.parquet	Only cleaning, no modelling decisions - the rules can be reviewed by a business reader
2 Feature engineering	features/build_features.py	train.parquet -> preprocessor.joblib	One function, <code>feature_frame()</code> , is used by training, batch inference and the serving client, so the three cannot drift apart
3 Training	models/train_model.py	train/test -> MLflow runs, registered model	Preprocessing and estimator are one sklearn Pipeline, logged as a single model
4 Inference	models/predict_model.py	any cleaned table -> predictions.csv	Loads the model by registry alias, not by file path; adds a risk band and a rank

Each step is a module with a `main()` function and an `argparse` interface, so the same code is reachable four ways: `python -m churn.<step>`, a Make target, a Poetry script (`churn-data`, `churn-features`, `churn-train`, `churn-predict`) and an MLflow entry point. **config.py** is the only place where paths, column lists and constants are written - no filename appears twice in the project.

5. Standardization

Python version

The group standard is **Python 3.11 or newer**, declared in `pyproject.toml` (`python = ">=3.11,<3.15"`). CI enforces the lower bound by running on 3.11; the project was developed and tested locally on 3.14. Version ranges were chosen instead of exact caret pins after a real problem: a pin of `pyarrow ^17` forced pip to build `pyarrow` from source on Python 3.14, where no pre-built package of that version exists (see section 11).

PEP8 tooling

Tool	Role	Configuration	Result
black	Formatter - removes style debates	line length 100 (pyproject.toml)	16 files unchanged
flake8	PEP8 checker	max line 100, E203/W503 ignored for black compatibility (setup.cfg)	0 warnings
pylint	Static analysis: naming, docstrings, complexity	.pylintrc	10.00 / 10

The one pylint finding during development was instructive: `train_one()` took six positional arguments. Rather than silencing the rule, the optional ones (`run_name`, `nested`) were made keyword-only, which also makes every call site self-describing.

Docstrings

Every public module and function has a Google-style docstring (Args / Returns). They are written for the reader who needs the **why** - for example, why the split is stratified or why the served model returns probabilities - and they are what Sphinx renders into the API reference.

6. Reproducibility

Layer	Mechanism
Dependencies	Poetry (pyproject.toml) is the reference. <code>requirements.txt</code> and <code>requirements-dev.txt</code> mirror it for plain pip and for CI; <code>python_env.yaml</code> points MLflow at the same list
Environment isolation	A virtual environment per clone (<code>.venv</code> , git-ignored); MLflow projects run with <code>--env-manager local</code> or build their own <code>virtualenv</code> from <code>python_env.yaml</code>
Randomness	A single <code>RANDOM_STATE = 42</code> in <code>config.py</code> feeds the split and every estimator
Data	Raw file committed; every downstream file is regenerated by <code>make data</code> / <code>make features</code>
Model inputs	The MLflow model signature records the 23 input columns and their types; numeric inputs are cast to float so a missing value never breaks schema enforcement
Execution	make all rebuilds everything from the raw file; <code>mlflow run . -e main</code> does the same through MLflow

Reproducibility, observed

The default LightGBM run and the sweep run with `learning_rate=0.05`, `num_leaves=31` use identical settings. They were trained separately and produced identical metrics to the last digit (PR-AUC 0.99901, Brier 0.00449) - the fixed seed and the deterministic pipeline doing their job.

Getting started from a clone

```
git clone https://github.com/ribbzz/AI_Project_Methodology.git
cd AI_Project_Methodology
poetry install && poetry shell      # or: python -m venv .venv && make install
make all                          # data -> features -> train + sweep + register -> predict
make ui                           # MLflow UI on http://127.0.0.1:5000
```

7. Documentation

Documentation is generated with **Sphinx** from the code itself, so it cannot fall out of date with the docstrings. `make docs` builds it into `docs/build/` (git-ignored); CI builds it with warnings treated as errors.

Component	Choice
Extensions	autodoc (API from docstrings), napoleon (Google style), autosummary, viewcode (links to highlighted source), intersphinx (links to Python, pandas, scikit-learn docs)
Theme	sphinx_rtd_theme
Pages	index - project overview and the four steps; usage - installation, pipeline, MLflow project, serving, quality gates; api - every module of the package
Heavy imports	mlflow and lightgbm are mocked (autodoc_mock_imports), so the docs build fast and without the ML stack

8. Model tracking and deployment with MLflow

MLflow 3 is used for all four capabilities named in the brief. The tracking store is a SQLite database (`mlflow.db`) plus an artefact directory (`mlruns/`), both pinned to the project root in `config.py` so that every script writes to the same place whatever directory it is launched from.

8.1 Tracking - parameters, metrics and multiple runs

Every call to `train_one()` opens a run and records:

Logged item	Content
Parameters	model family, decision threshold, train/test sizes, and every scalar hyper-parameter of the estimator (prefixed model)
Metrics	<code>pr_auc</code> , <code>roc_auc</code> , <code>lift_at_10pct</code> , <code>recall_at_10pct</code> , precision, recall, f1, brier
Tags	dataset file, model family; the sweep parent also records its grid and best child
Artefacts	<code>plots/confusion.png</code> , <code>curves.png</code> (ROC + precision-recall), <code>lift.png</code> , <code>importance.png</code>
Model	the whole preprocessing + estimator pipeline, with signature and input example

The experiment **ecommerce-churn** holds ten runs: three model families (logistic regression, random

forest, LightGBM) and a LightGBM sweep over learning_rate x num_leaves, logged as six **nested runs** under one parent so that the UI groups them.

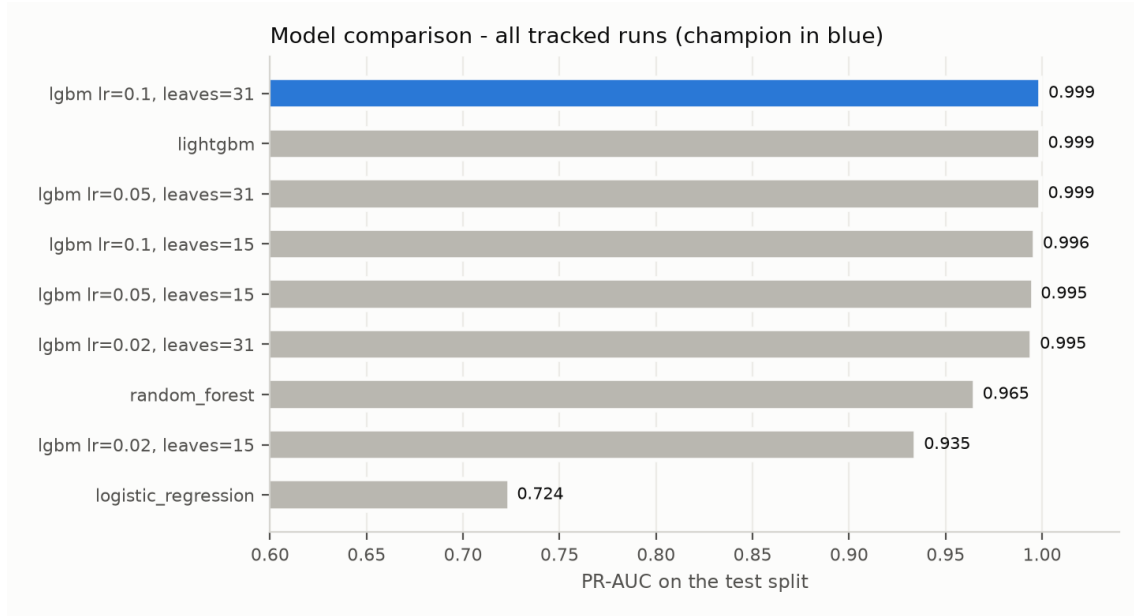


Figure 2 - PR-AUC of every tracked run, generated from the tracking store by scripts/report_figures.py.

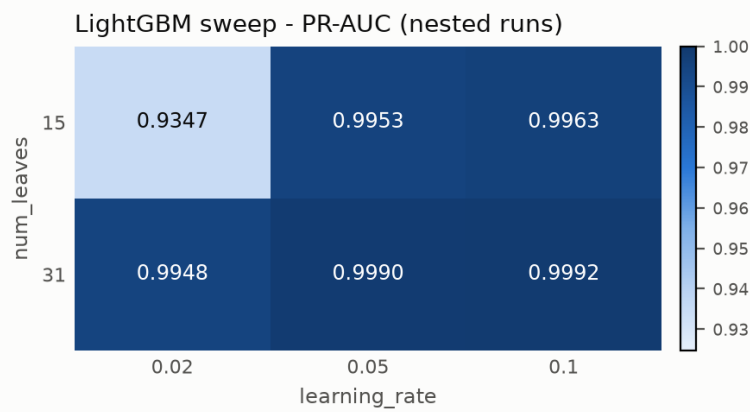
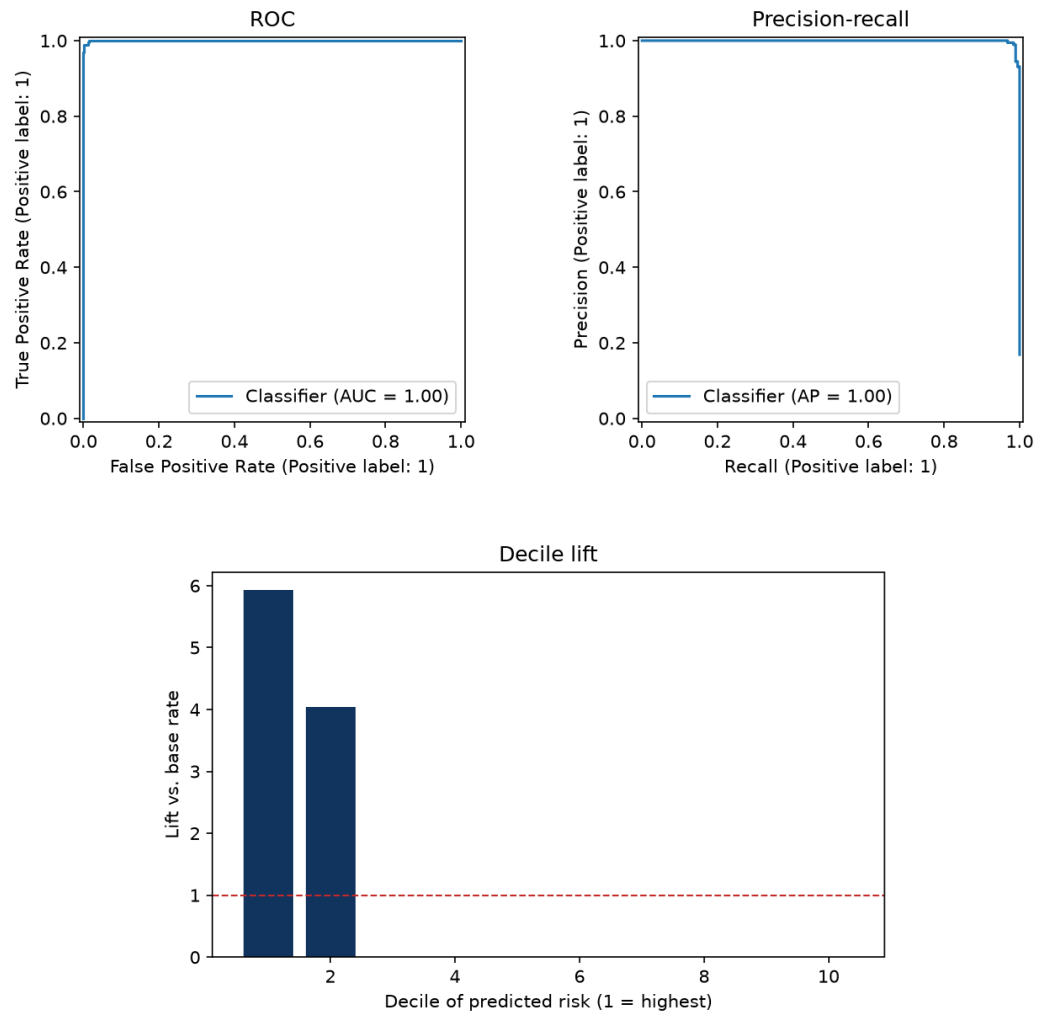


Figure 3 - The nested LightGBM sweep. Small trees with a slow learning rate under-fit within 600 iterations; the other settings are within 0.005 of each other.

Run	PR-AUC	ROC-AUC	Lift @10%	Recall @10%	Precision	Recall	Brier
LightGBM lr=0.1, leaves=31	0.9992	0.9998	5.93	0.59	0.99	0.99	0.0035
LightGBM (defaults)	0.9990	0.9998	5.93	0.59	0.98	0.98	0.0045
LightGBM lr=0.1, leaves=15	0.9963	0.9993	5.93	0.59	0.97	0.99	0.0052
LightGBM lr=0.05, leaves=15	0.9953	0.9991	5.93	0.59	0.91	1.00	0.0152
LightGBM lr=0.02, leaves=31	0.9948	0.9990	5.93	0.59	0.91	0.99	0.0173
Random forest	0.9651	0.9928	5.87	0.58	0.88	0.87	0.0372
LightGBM lr=0.02, leaves=15	0.9347	0.9884	5.56	0.55	0.79	0.96	0.0435
Logistic regression	0.7239	0.9035	4.55	0.45	0.52	0.84	0.1181

Test split, 1,126 customers, threshold 0.5. The LightGBM lr=0.05, leaves=31 sweep run is identical to the defaults run and is omitted.

Evaluation plots of the champion run (MLflow artefacts)



Figures 4-5 - ROC and precision-recall curves; decile lift. The first decile reaches the maximum possible lift on this split.

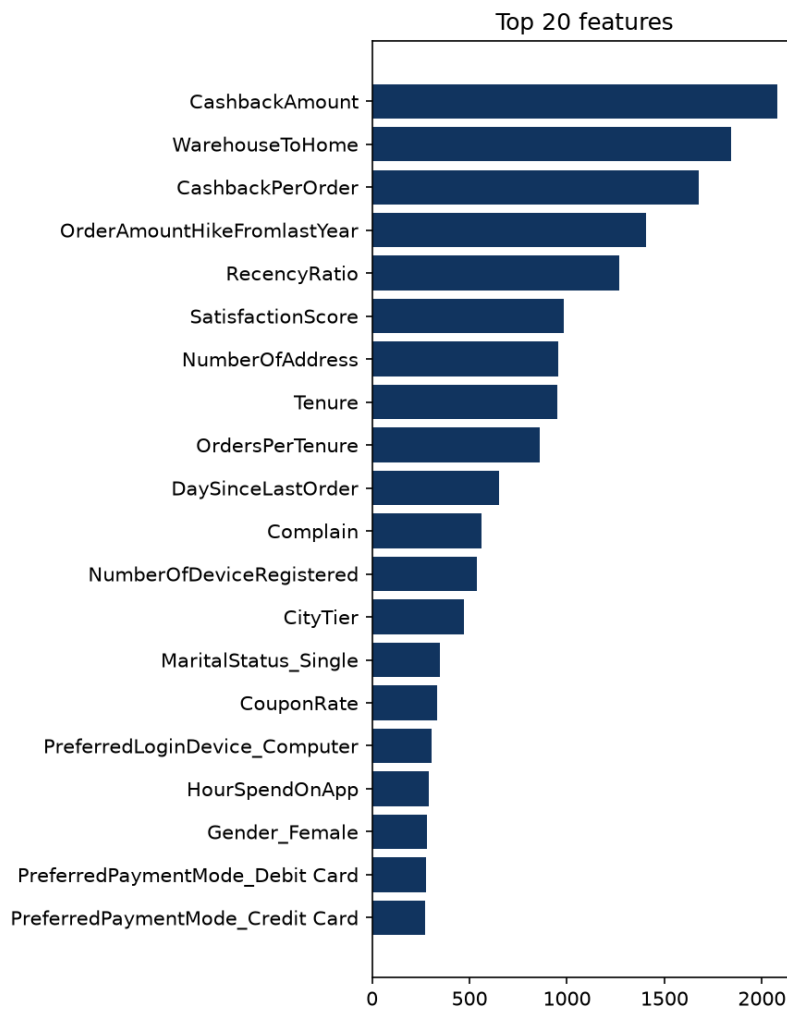


Figure 6 - Feature importance of the champion (LightGBM split counts). Tenure, cashback and the engineered ratios lead.

8.2 MLflow Projects - packaging

The repository is an MLflow project. The MLproject file names the environment (python_env.yaml) and seven entry points, so the whole pipeline can be run by someone who only has MLflow installed:

Entry point	Parameters	Command
prepare	-	python -m churn.data.make_dataset
features	-	python -m churn.features.build_features
train	model (lightgbm), threshold (0.5)	train_model --model {model} --threshold {threshold}
sweep	-	train_model --sweep
train_all	-	train_model --all --sweep --register
predict	input_path, alias (champion)	predict_model --input {input_path} --alias {alias}
main	-	prepare, features and train_all chained

```
mlflow run . -e main --env-manager local
mlflow run . -e train -P model=random_forest -P threshold=0.4 --env-manager local
```

8.3 Model Registry - versioning and management

After training, `register_best()` picks the run with the highest PR-AUC and registers its model as **ecommerce-churn-classifier**. The version is given the alias **champion**, two tags (`selected_on = pr_auc`, `run_name`) and a description with its scores.

Registry element	Value
Registered model	ecommerce-churn-classifier
Version 1	LightGBM, learning_rate=0.1, num_leaves=31 - PR-AUC 0.9992, lift@10% 5.93
Alias	champion -> version 1
Lineage	the version links back to its run: parameters, metrics, plots and code are one click away

Aliases rather than stages. MLflow 3 deprecates the fixed None / Staging / Production / Archived stages in favour of aliases. Consumers load models:/ecommerce-churn-classifier@champion; promoting a new model, or rolling back, is a single alias move with no code change and no redeployment - the champion / challenger practice from Part 1 made concrete.

8.4 Serving - local inference server

The champion is served over REST by MLflow's inference server (FastAPI + uvicorn). make serve first points the store at the current checkout, then starts it:

```
MLFLOW_TRACKING_URI=sqlite:///mlflow.db \
  mlflow models serve -m "models:/ecommerce-churn-classifier@champion" -p 5001 --env-manager local

python scripts/score_request.py --port 5001      # POST /invocations
```

The model is logged with **pyfunc_predict_fn="predict_proba"**, so the endpoint returns [p(stay), p(churn)] for each customer instead of a hard 0/1. A label cannot be ranked, and ranking is exactly what the retention team needs (Part 1, section 4.3). The client sends three churners and three non-churners taken from the test split:

CustomerID	actual_churn	churn_probability
52126	1	0.9999
50959	1	0.9999
55217	1	0.9998
54007	0	0.0000
51970	0	0.0000
54236	0	0.0000

8.5 Batch inference

make predict loads the champion by alias, scores the test split and writes reports/predictions.csv with the probability, a **risk band** and the rank. Bands use the thresholds in config.py (HIGH ≥ 0.60 , MEDIUM ≥ 0.35):

Band	Customers	Actual churn rate
HIGH	190	98.9%
MEDIUM	2	0.0%
LOW	934	0.2%

9. Reading the results critically

A PR-AUC of 0.999 would be a red flag in production. Part 1 insisted on scepticism towards scores that look too good, so the same scepticism is applied here.

Observation	Explanation	What it means
Near-perfect separation	The Kaggle dataset is a clean, static extract with strong, low-noise signals (tenure, complaints, cashback). Tree ensembles capture their interactions almost completely	Good for demonstrating the tooling; not evidence that churn is this predictable in reality
No out-of-time validation	The file has no date column, so the time-based split recommended in Part 1 is impossible; the stratified random split is the best available	Scores are optimistic. With real data the split would be by scoring date
Possible leakage cannot be ruled out	The data dictionary does not state when columns such as DaySinceLastOrder or CashbackAmount were captured relative to the churn label	In a real project these would go through the leakage checklist of Part 1 before any deployment
Lift @10% stops at 5.93	The top decile (112 customers) is smaller than the churning population (190), so a perfect ranking gives a lift of $1 / 0.169 = 5.93$	Metric saturated; recall @10% (0.59) is capped the same way. A 20% cutoff would discriminate better
Logistic regression lags (PR-AUC 0.72)	The signal is in interactions and thresholds that a linear model cannot represent without manual feature engineering	Still the transparent baseline Part 1 asked for; the gap measures what the tree model adds

10. Testing and quality

21 pytest tests cover the whole pipeline. They run in CI and with make test:

Area	What is tested
Data preparation	category spellings merged; duplicate customers dropped; split sizes and stratification
Features	engineered columns finite, including tenure = 0; unseen categories and missing values handled by the fitted preprocessor
Metrics	lift and recall at k on a perfect ranking; metric set and values
Training	every candidate pipeline fits and returns probabilities; parameter overrides applied; a run logs parameters, metrics and plots; registration sets the champion alias
Inference	risk band thresholds; one row per customer with a unique rank
Plots	all four evaluation figures are written

Training tests use a throw-away MLflow store in a temporary directory, so running the suite never adds runs to the project's experiment. Line coverage is **76%**; what remains uncovered is mostly the command-line entry points, which make all exercises end to end.

11. Technical challenges encountered

The brief is about the technical challenges of AI projects. These are the ones this project actually hit, with their fixes - each is visible in the code.

Problem	Cause	Fix
Logging the model failed: 'untrusted types found: numpy.dtype'	MLflow 3.16 serialises sklearn models with skops by default, and skops refuses some types present in a LightGBM pipeline	serialization_format="cloudpickle" in log_model, with a comment explaining why
The REST endpoint returned 0/1 labels	The pyfunc wrapper calls predict() by default	pyfunc_predict_fn="predict_proba"; the signature is inferred from the probabilities
mlflow models serve exited immediately	MLflow 3 serves through uvicorn / FastAPI, which were not dependencies	uvicorn and fastapi added to the runtime dependencies
MLflow warned that integer columns cannot hold missing values	The signature was inferred with some numeric columns typed as integers	feature_frame() casts every numeric input to float64
Committed MLflow outputs break after a clone	MLflow stores artefact locations as absolute paths	scripts/relocate_mlflow.py rewrites them to the current checkout; run by make ui / make serve
Scripts wrote to different stores depending on the working directory	MLflow's default tracking location is relative to the current directory	TRACKING_URI and the artefact root are pinned in config.py
pip tried to compile pyarrow from source	Caret pins (pyarrow ^17) excluded the newer versions that ship pre-built packages for Python 3.14	Version ranges with an upper bound on the major version only
The MLflow UI exhausted memory on a small machine	mlflow ui starts four worker processes by default (about 230 MB each)	make ui passes --workers 1

12. Conclusion

The project delivers every item of the brief: versioned code on GitHub with CI, a cookiecutter structure with one script per pipeline step, PEP8 tooling passing cleanly, Poetry-managed dependencies, generated Sphinx documentation, and MLflow used end to end - ten tracked runs, a packaged MLflow project, a registered champion model and a working local REST endpoint.

It also closes the loop with Part 1. The choices framed there - rank-based metrics for an imbalanced target, one transformation path shared by training and serving, a registry that makes promotion and rollback a single step, probabilities and risk bands instead of bare labels, and scepticism towards scores that look too good - are now implemented in code, not only described.

13. References

- Verma, A. **E-Commerce Customer Churn Analysis and Prediction** (dataset). Kaggle. kaggle.com/datasets/ankitverma2010/ecommerce-customer-churn-analysis-and-prediction
- DrivenData. **Cookiecutter Data Science**. drivendata.github.io/cookiecutter-data-science/
- van Rossum, G., Warsaw, B., Coghlan, N. **PEP 8 - Style Guide for Python Code**. peps.python.org/pep-0008/
- MLflow documentation - **Tracking, Projects, Model Registry, Deployment**. mlflow.org/docs/latest/ml/
- Sphinx documentation. sphinx-doc.org/en/master/ ; napoleon extension for Google-style docstrings.
- Poetry documentation. python-poetry.org/docs/
- black, flake8 and pylint documentation (pypi.org/project/black, [/flake8](https://pypi.org/project/flake8/), [/pylint](https://pypi.org/project/pylint/)).
- Pedregosa, F. et al. (2011). **Scikit-learn: Machine Learning in Python**. JMLR 12.
- Ke, G. et al. (2017). **LightGBM: A Highly Efficient Gradient Boosting Decision Tree**. NeurIPS.
- Saito, T. and Rehmsmeier, M. (2015). **The precision-recall plot is more informative than the ROC plot on imbalanced datasets**. PLoS ONE.
- GitHub Actions documentation. docs.github.com/actions
- Group report, **Graded Project - Part 1: functional framing of the RetailGenius churn project**, 2026.

Use of generative AI

Generative AI (Claude, Anthropic) was used as an assistant to help write and debug code, structure this report and review the project against the brief. All design choices, results and text were reviewed and validated by the group.